

Watchdog Timer

Watchdog Timer

- 1. Watchdog Overview
 - How Important is the Watchdog? Real-World Lessons
- 2. Core Concepts
 - 2.1 Watchdog Principle
 - 2.2 Non-Blocking Delay
 - 2.3 `millis()` Overflow Handling
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Macros and Global Variables
 - 5.2 Feed and Timeout Detection
 - 5.3 Non-Blocking Delay + Button Detection
 - 5.4 `setup()` — Three-Phase Demo
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Behavior
 - 6.3 Boundary Tests
- 7. Common Issues

1. Watchdog Overview

Watchdog Timer (WDT) is a fault tolerance mechanism: the program must "**feed the dog (reset)**" within a specified time, otherwise the watchdog determines the system is frozen and automatically resets. This experiment uses software to simulate a watchdog, demonstrating in three phases: automatic feeding (normal operation) → stop feeding (simulating freeze) → timeout reset. The reason for using software simulation instead of hardware WDT is that hardware WDT resets too fast to print debug information; software simulation allows clear observation of the entire process.

Typical Application Scenarios:

Application Type	Example
Industrial Control	PLC freeze protection, production line controller automatic recovery
Remote Devices	Unattended base stations, outdoor smart meters (electricity/water)
Communication Equipment	Router/gateway deadlock auto-reset
Medical/Automotive	Patient monitors, infusion pumps, automotive ECU (mandatory safety certification)
Smart Home	Ceiling-mounted gateways, smart locks (unreachable by humans, can only self-recover)

How Important is the Watchdog? Real-World Lessons

The watchdog is not a "premium feature" but the **bottom line for survival** of embedded products. Here are some real scenarios:

- **Industrial Controller (PLC):** If one controller on the production line crashes, the entire assembly line stops. Losses could reach thousands of yuan per minute. With WDT, auto-restart recovers production within seconds; without WDT, you wait for a worker to run over and press the reset button. If no one is around, losses are tens of thousands of yuan per hour.
- **Smart Meters (Electricity/Water):** Installed in outdoor pipe wells or on power poles. Once it crashes, no one knows. A month of data is lost. WDT is its only means of "self-rescue."
- **Drone Flight Controller:** If the program runs away while flying, no WDT means free fall; with WDT, the flight controller can at least restart instantly and switch to safe mode for landing.
- **Medical Equipment:** If a patient monitor or infusion pump crashes, it's not a "just restart it" problem. It's a matter of life and death. WDT is a **mandatory requirement** for medical certification.
- **Automotive ECU:** The engine control unit crashes while driving? Without WDT, you don't even want to think about it. So automotive-grade MCU watchdogs use independent clock sources, completely decoupled from the main CPU.
- **Smart Home Gateway:** Installed in ceiling cavities. If it crashes, humans can't reach it. Without WDT, it's just a brick. Users can only power cycle it. Terrible experience.

In short: As long as your device is not connected to a debugger everywhere you go, you should enable the watchdog. The watchdog is not used to prevent bugs, but to prevent the consequences of bugs — device becoming a brick, production lines stopping, even casualties. It's a "worst-case insurance."

2. Core Concepts

2.1 Watchdog Principle

Normal Flow: Program periodically "feeds the dog" (resets counter)
feed() → feed() → feed() → ...

Abnormal Flow: Program freezes → Stop feeding → Counter overflow → System reset

This experiment uses `millis()` and `last_feed_time` variable to simulate the hardware watchdog counter mechanism: `millis() - last_feed_time > 5s` determines timeout.

2.2 Non-Blocking Delay

The program needs to continuously check buttons and timeout while waiting, without using `delay()`. This experiment uses `delay_check_button()` function to split long delays into 20ms fragments, checking button and timeout once per fragment:

```
while (not time yet) {
  if (button pressed) feed;
  if (timeout) return;
  delay(20ms); // short fragment wait
}
```

2.3 millis() Overflow Handling

millis() returns unsigned long, which will overflow back to 0 after approximately 49 days. Using unsigned subtraction is naturally immune to overflow:

```
millis() - last_feed_time > WDT_TIMEOUT_MS
```

Even at overflow boundaries (e.g., millis()=100, last_feed_time=4294967200), unsigned subtraction's modulo arithmetic still gives the correct difference. This is a classic technique in embedded systems.

3. Hardware Dependencies

Pin	Peripheral	Configuration	Description
GPIO0	BOOT Button	INPUT_PULLUP	Press = low level, used for manual feed

4. Project Architecture and Flow

```
setup()
├─ Initialize serial + button
├─ feed_watchdog()           // Initial feed
├─
├─ Phase 1: Auto Feed        // Simulate normal operation
├─   └─ for i=1..5:
├─     └─ feed() + delay_check_button(1000ms)
├─
├─ Phase 2: Stop Feed        // Simulate freeze
├─   └─ Countdown 3 seconds, BOOT can extend feed during this time
├─
├─ Phase 3: Timeout Reset
├─   └─ ESP.restart()

delay_check_button(ms)       // Non-blocking wait
├─ while (not ms yet):
├─   └─ BOOT pressed? → feed() + wait for release → return 1
├─   └─ Timeout? → return 2
```

5. Source Code Details

Full source code: [Source Code](#) → [Arduino](#) → [1.Basic Peripherals](#) → [6.Watch_Dog](#) → [6.watch_Dog.ino](#)

5.1 Macros and Global Variables

Configure watchdog timeout, feed count, countdown and other parameters, all centralized in macros for easy adjustment.

```
#define BOOT_PIN          0
#define FEED_COUNT        5           // Phase 1 auto feed 5 times
#define FEED_INTERVAL_MS 1000        // 1s interval between feeds
#define COUNTDOWN_SEC     3           // 3s countdown after stopping feed
#define WDT_TIMEOUT_MS    5000       // 5s without feed = timeout

unsigned long last_feed_time = 0;     // Last feed timestamp
bool wdt_enabled = true;
```

5.2 Feed and Timeout Detection

`feed_watchdog()` records the timestamp; "feeding the dog" means telling the watchdog "I'm still alive".

`watchdog_timeout()` checks for timeout.

```
void feed_watchdog() {
    last_feed_time = millis();
}

bool watchdog_timeout() {
    return wdt_enabled && (millis() - last_feed_time > WDT_TIMEOUT_MS);
}
```

`millis() - last_feed_time` uses unsigned subtraction, still correct even crossing the ~49 day overflow point.

5.3 Non-Blocking Delay + Button Detection

This is the core of the entire experiment — splitting long delays into 20ms fragments while maintaining real-time responsiveness.

[illegible]

The three return values correspond to three events: nothing happened / manual feed / timeout. The caller decides subsequent actions based on the return value.

5.4 `setup()` — Three-Phase Demo

All logic runs once in `setup()`, `loop()` is left empty.

```
void setup() {
  Serial.begin(115200);
  pinMode(BOOT_PIN, INPUT_PULLUP);
  feed_watchdog();

  // === Phase 1: Auto Feed ===
  for (int i = 1; i <= FEED_COUNT; i++) {
    feed_watchdog();
    Serial.printf("[WDT] Auto Feed #%d/%d\n", i, FEED_COUNT);
    if (delay_check_button(FEED_INTERVAL_MS) == 2) break;
  }

  // === Phase 2: Stop Feed ===
  int countdown = COUNTDOWN_SEC;
  while (countdown > 0) {
    int ret = delay_check_button(1000);
    if (ret == 1) { countdown = COUNTDOWN_SEC; } // BOOT pressed → reset countdown
    else if (ret == 2) { break; }                // Timeout → enter Phase 3
    else { countdown--; }
  }

  // === Phase 3: Reset ===
  Serial.println("\n[WDT] Watchdog timeout! Resetting...");
  ESP.restart();
}

void loop() {}
```

Pressing BOOT in Phase 2 resets the countdown — equivalent to "rescuing" a frozen system, demonstrating the typical interaction pattern for manual watchdog intervention.

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Normal Behavior

Open the **Serial Monitor** (115200 bps) and observe the three phases:

- Phase 1: Auto feed once every 1 second, 5 times total
- Phase 2: Stop feeding, 3 second countdown. Press BOOT to reset countdown

-

```
=====
ESP32-S3 Watchdog Test (Arduino)
Press BOOT button to manual feed
=====
Watchdog started, timeout: 5000 ms

=== Phase 1: Auto Feeding ===
[WDT] Auto Feed #1/5 | Timer: 138 ms
[WDT] Auto Feed #2/5 | Timer: 1157 ms
[WDT] Auto Feed #3/5 | Timer: 2157 ms
  >> [BTN] Manual feed! Timer reset. | 2677 ms
[WDT] Auto Feed #4/5 | Timer: 2827 ms
[WDT] Auto Feed #5/5 | Timer: 3827 ms

=== Phase 2: Stop Auto Feed ===
Feeding stopped! WDT resets in 5000 ms.
Press BOOT to manual feed and extend time!

[WDT] Countdown 3 ...
[WDT] Countdown 2 ...
  >> [BTN] Manual feed! Timer reset. | 6158 ms
  >> Countdown restarted to 3s!
[WDT] Countdown 3 ...
[WDT] Countdown 2 ...
[WDT] Countdown 1 ...
  >> [BTN] Manual feed! Timer reset. | 8728 ms
  >> Countdown restarted to 3s!
[WDT] Countdown 3 ...
[WDT] Countdown 2 ...
[WDT] Countdown 1 ...
  >> [BTN] Manual feed! Timer reset. | 11418 ms
  >> Countdown restarted to 3s!
[WDT] Countdown 3 ...
[WDT] Countdown 2 ...
[WDT] Countdown 1 ...
  >> [BTN] Manual feed! Timer reset. | 14108 ms
  >> Countdown restarted to 3s!
```

- Phase 3: Timeout, chip restarts after printing "Resetting..."

```
  >> [BTN] Manual feed! Timer reset. | 16198 ms
  >> Countdown restarted to 3s!
[WDT] Countdown 3 ...
[WDT] Countdown 2 ...
[WDT] Countdown 1 ...

[WDT] Watchdog timeout! Resetting system...
```

6.3 Boundary Tests

Test Case	Operation	Expected Result
No button press	Don't touch BOOT at all	Phase 1 → Phase 2 → 3 second countdown → Reset

Test Case	Operation	Expected Result
Press BOOT during Phase 2	Press once every 2 seconds	Countdown repeatedly resets, system doesn't restart
Long press BOOT	Hold BOOT during Phase 2	One feed + wait for release, countdown resets once

7. Common Issues

Symptom	Cause	Solution
Resets in Phase 2 without timeout	Countdown logic or <code>delay_check_button</code> timeout detection conflict	Check return value handling
Continuous feeding when button held	Not waiting for release	Confirm <code>while (digitalRead(BOOT_PIN) == LOW) delay(10)</code> exists
Garbled serial output	Baud rate mismatch	Confirm monitor set to 115200